University of Southern Denmark

Department of Mathematics and Computer Science

SPD801: Master Thesis in Computer Science

Formalising πLL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

21st May 2026

1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π-calculus.

This line of work has been further developed by [Montesi and Peressotti 2021], who provide a unified account reconciling linear logic, the π-calculus, and their metatheory. Their framework serves as the formal basis for the system πLL considered in this thesis.

Motivation and correctness concerns. Despite the elegance of these theoretical correspondences, their informal and paper-based presentations are prone to subtle errors. In particular, the application of inference rules in linear logic and session-typed systems often relies on implicit ordering conventions and intuitive interpretations of syntax. This can lead to mistakes that are not immediately apparent in written derivations.

As an illustrative example, small inconsistencies can arise in the presentation of typing rules within such systems, where the intuitive alignment between variable ordering and type assignment does not precisely match the formal rule structure. These discrepancies are not flaws of the underlying theory, but rather reflect the difficulty of maintaining precision in informal presentations, including in the framework of Montesi and Peressotti.

As observed in discussions with the authors of the framework, such issues are easy to overlook because rule application is often guided by intuition rather than strict syntactic structure. This highlights that even formally correct rules may be unintuitive to apply due to their representation.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanization. Mechanization provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanization in this work is to validate and internalize existing results within a proof assistant. This allows us to eliminate ambiguity arising from informal presentation and to ensure that all derivations are checked with respect to a precise formal semantics.

Moreover, mechanized developments have repeatedly demonstrated their ability to uncover small but meaningful inconsistencies in paper proofs, supporting the view that proof assistants serve as a valuable tool for verifying, rather than merely implementing, theoretical frameworks.

Contributions. In this thesis, we present a mechanized formalization of the π-calculus-based linear logic system πLL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalization of the syntax and typing system of πLL in Lean, including processes, session types, and substitution operations.
- A formalization of the operational semantics for the multiplicative fragment of πLL.
- Formal proofs of session fidelity for the multiplicative fragment.
- A locally nameless representation of binding, eliminating the need for reasoning up to α-equivalence and simplifying substitution reasoning.

Scope of the formalization. This thesis focuses on a mechanized reconstruction of the πLL system of [Montesi and Peressotti 2021], with particular emphasis on its multiplicative fragment. The

syntactic and typing components of πLL have been fully mechanized, including processes, session types, typing judgments, and substitution operations. The operational semantics are defined for the multiplicative fragment, with the exception of the res-rule.

Consequently, metatheoretic results concerning typing apply to the full system, while results involving operational semantics are restricted to the multiplicative fragment. The aim of this work is not to extend the theory, but to provide a mechanically verified account of an existing theoretical framework.

2 Background

This section begins by introducing the theoretical foundations underlying the formalisation following the presentation of [Montesi and Peressotti 2021], We cover the full πLL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment πMLL when discussing operational semantics.

Following [Montesi and Peressotti 2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

In particular, conjunction and disjunction split into multiplicative and additive variants. The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \parr B$, together with the corresponding units 1 and $\bot$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the joint availability of two independent resources, while $A \parr B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B, whereas $A \parr B$ corresponds to receiving a value of type A and proceeding as B. The units 1 and $\bot$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1}\,1 \qquad \frac{\vdash \Gamma}{\vdash \Gamma, \bot}\,\bot \qquad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B}\,\otimes \qquad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \parr B}\,\parr$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B, each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B.

The unit rule rule 1 introduces the unit 1, which represents the absence of further obligations. It can be understood as an "empty output", corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\bot$ and $\parr$ describe the behavior of the connective $A \parr B$ and its unit $\bot$. The $\parr$ rule combines resources within a single context, reflecting a form of interaction where both components

are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B. The unit $\bot$ represents the absence of further input, acting as an "empty input" and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the rule 1 and combined via rule $\otimes$ to derive $1 \otimes 1$:

$$\dfrac{\dfrac{}{\vdash 1}\,1 \qquad \dfrac{}{\vdash 1}\,1}{\vdash 1 \otimes 1}\otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-oriented view of logic provides the foundation for its applications in computer science, particularly in the study of concurrency and session-typed communication. In the following sections, this perspective is used to motivate the process interpretation underlying πLL.

2.2 The π-Calculus: Processes and Communication

The π-calculus [Milner et al. 1992] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of Montesi and Peressotti [2021], which adopts Wadler's convention of using square brackets '[-]' for output and round parentheses '(-)' for input.

Programs in πMLL are processes (P, Q, R, ...), which communicate over names (x, y, z, ...) representing session endpoints. Sessions are binary (they involve exactly two endpoints), a discipline enforced by the typing system introduced in 2.3. We assume an infinite set of names with decidable equality. The process terms of πMLL are defined by the following grammar:

$P, Q ::=$	$x[y].P$	*output y on x and continue as P*
$\mid$	$x(y).P$	*input y on x and continue as P*
$\mid$	$x[].P$	*output (empty message) on x and continue as P*
$\mid$	$x().P$	*input (empty message) on x and continue as P*
$\mid$	$\boldsymbol{\nu} xy\,P$	*name restriction (cut)*
$\mid$	$P \mid Q$	*parallel composition*
$\mid$	$x[\mathrm{L}].P$	*select left on x and continue as P*
$\mid$	$x[\mathrm{R}].P$	*select right on x and continue as P*
$\mid$	$x.\mathrm{case}\{\mathrm{L}:P, \mathrm{R}:Q\}$	*offer a binary choice between P or Q on x*
$\mid$	$x[A].P$	*output type A on x and continue as P*
$\mid$	$x(X).P$	*input a type as X on x and continue as P*
$\mid$	$!x\langle z_1, \ldots, z_n\rangle.\{P\}$	*server (replicable process) on x depending on servers on $z_1 \ldots z_n$*
$\mid$	$x[\mathrm{USE}].P$	*consume a server on x and continue as P*
$\mid$	$x[\mathrm{DUP}](x').P$	*duplicate server x as x' in P*
$\mid$	$x[\mathrm{DISP}].P$	*dispose of a server on x*
$\mid$	$x \leftrightarrow y$	*link x and y*
$\mid$	$\mathbf{0}$	*terminated process*

Term $x[y].P$ allocates a fresh name y, outputs it over x, and proceeds as P. Dually, $x(y).P$ inputs a name over x, binding it to y in P. Terms $x[].P$ and $x().P$ model empty output and input. Restriction

$\boldsymbol{\nu} xy\,P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\boldsymbol{\nu} xy\,P = \boldsymbol{\nu} yx\,P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and $\mathbf{0}$ is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \boldsymbol{\nu} xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0})$. This process creates a private session between the restricted endpoints x and z. The left process, $x[].\mathbf{0}$, signals termination on x before terminating, while the right process, $z().y[].\mathbf{0}$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and Propositions-as-Sessions Correspondence

In πLL, session types are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality, the key structural feature of classical linear logic, ensures that the two endpoints of every session implement complementary protocols.

The correspondence between linear logic and session types as presented in [Montesi and Peressotti 2021] following [Carbone et al. 2016; Wadler 2014] is captured directly by the type grammar. Each connective denotes a communication action, where $A \otimes B$ describes sending a value of type A and continuing as B, while its dual $A \parr B$ describes receiving. The units $\mathbf{1}$ and $\bot$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \mathbin{\&} B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined as follows:

$A, B ::=$	$A \otimes B$	*send A, proceed as B*	$\mid$	$A \parr B$	*receive A, proceed as B*
$\mid$	$\mathbf{1}$	*empty output, unit for* $\otimes$	$\mid$	$\bot$	*empty input, unit for* $\parr$
$\mid$	$A \mathbin{\&} B$	*offer A or B*	$\mid$	$A \oplus B$	*select A or B*
$\mid$	$\exists X.A$	*existential, type output*	$\mid$	$\forall X.A$	*universal, type input*
$\mid$	$?A$	*client request*	$\mid$	$!A$	*server accept*
$\mid$	X	*type variable*	$\mid$	${}^{\bot}X$	*dual of type variable*

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, i.e. $(A^{\bot})^{\bot} = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$(A \otimes B)^{\bot} = A^{\bot} \parr B^{\bot} \qquad \mathbf{1}^{\bot} = \bot \qquad (A \oplus B)^{\bot} = A^{\bot} \mathbin{\&} B^{\bot}$$
$$(!A)^{\bot} = ?A^{\bot} \qquad (\exists X.A)^{\bot} = \forall X.A^{\bot} \qquad (X)^{\bot} = X^{\bot}$$

2.4 Environments, Hyperenvironments and Judgements

Typing in πLL is organised into two layers. Environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in [Montesi and Peressotti 2021], *environments* ($\Gamma, \Delta, \Xi, \ldots$) are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A, \ldots, x_n : A_n$, where each x_i is associated to its respective A_i, for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then Γ, Δ is the

environment containing exactly the associations in Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using "," as the sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \qquad \Gamma, \Delta = \Delta, \Gamma \qquad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by handled by *hyperenvironments* ($\mathcal{G}, \mathcal{H}, \mathcal{I}, \ldots$), which are collections of environments joined using the parallel operator '$\mid$'. They can be written as follows $\mathcal{G} = \Gamma_1, \ldots, \Gamma_n$, where $\Gamma_1, \ldots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} \mid \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using '$\mid$' the sum and $\varnothing$ as unit, where $\varnothing$ is the hyperenvironment containing no environments:

$$\mathcal{G} \mid \varnothing = \mathcal{G} \qquad \mathcal{G} \mid \mathcal{H} = \mathcal{H} \mid \mathcal{G} \qquad (\mathcal{G} \mid \mathcal{H}) \mid \mathcal{I} = \mathcal{G} \mid (\mathcal{H} \mid \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations ($\mathcal{D}, \mathcal{E}, \mathcal{F}, \ldots$) are written as $\overset{\mathcal{D}}{\vdash P :: \mathcal{G}}$ and are read as the derivation $\mathcal{D}$ having the judgement $\vdash P :: \mathcal{G}$ as its conclusion. The meaning of this statement is that the process, P, appearing in the judgement concluding a derivation is well-typed.

Multiplicative Fragment

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \qquad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \qquad \frac{}{\vdash \mathbf{0} :: \varnothing}\ \text{MIX}_0$$

$$\frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B}\ \otimes \qquad \frac{\vdash P :: \varnothing}{\vdash x[].P :: x : 1}\ 1 \qquad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \parr B}\ \parr \qquad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \bot}\ \bot$$

Additional rules for Additives, Quantifiers and Exponentials

$$\frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{L}].P :: \Gamma, x : A \oplus B}\ \oplus_1 \qquad \frac{\vdash P :: \Gamma, x : B}{\vdash x[\text{R}].P :: \Gamma, x : A \oplus B}\ \oplus_2 \qquad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{\text{L} : P, \text{R} : Q\} :: \Gamma, x : A \,\&\, B}\ \&$$

$$\frac{}{\vdash x \leftrightarrow y :: x : A^{\perp}, y : A}\ \text{AX} \qquad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A}\ ? \qquad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A}\ \text{W} \qquad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A}\ \text{C}$$

$$\frac{\vdash P :: z_1 : ?B_1, \ldots, z_n : ?B_n, x : A}{\vdash !x\langle z_1, \ldots, z_n \rangle.\{P\} :: z_1 : ?B_1, \ldots, z_n : ?B_n, x : !A}\ ! \qquad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B}\ \exists \qquad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B}\ \forall$$

Fig. 1. πLL, typing rules.

The multiplicative rules form the core of the system. Rule CUT composes two processes in P sharing dual endpoints $x : A$ and $y : A^{\perp}$, hiding them via restriction. This is the logical equivalent of communication. Rule MIX and rule MIX_0 compose independent processes in parallel and introduce the terminated process, respectively.

Rule $\otimes$ and rule $\parr$ type output and input. Note that rule $\otimes$ types the sent name y in a *separate* environment from the continuation, reflecting that the two are independent resources. By contrast, rule $\parr$ keeps y and the continuation x in the *same* environment, reflecting their sequential

dependency. The units rule 1 and rule $\bot$ type empty output and empty input, where rule 1 requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to $\mathbf{0}$.

The additive rules (rule $\oplus_1$, rule $\oplus_2$, and rule $\&$) type selection and offering of alternative processing paths. Note that rule $\&$ requires both branches to be typed in the *same* context Γ, reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The rule ! requires all free names except x to be typed using ?, enforcing that a server only depends on other servers. The rule ?, rule w, and rule c allow a client to use, discard, or duplicate a server respectively.

The rule Ax types a link between two dual endpoints, $x{\leftrightarrow}y$, forwarding all messages sent on x safely to y and vice versa. Rule $\exists$ and rule $\forall$ type polymorphic communication, where the side condition $X \notin \mathrm{ft}(\Gamma)$ in rule $\forall$, ensures the introduced type variable, X, does not accendentally shadow one already existing in Γ.

Example 2.2. The process from Example 2.1 has the typing $\vdash \nu zx\,(z().y[].\mathbf{0} \mid x[].\mathbf{0}) :: y : 1$, as shown by the derivation below:

$$\dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0}{\vdash y[].\mathbf{0} :: y : 1}\,1 \qquad \dfrac{}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}\,\mathrm{MIX}}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\mathrm{CUT}$$

Note that, for the typing context $x : 1 \mid y : 1, z : \bot$ to align with the premise of rule CUT, we utilise the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \varnothing$ and $\Gamma = \bullet$. Under these instantiations, the premise of rule CUT: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}$ can be viewed as $x : A \mid \Delta, y : A^{\perp}$. This effectively instantiates the CUT over the dual types $A = 1$ and $A^{\perp} = \bot$ on channels x and y, with $\Gamma = y : 1$, while ensuring the restricted endpoints implement dual session behaviours, fulfilling the requirements for a creating a valid CUT (session).

2.5 Operational Semantics

The operational semantics of πLL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in Figure 2. The full LTS is given in Montesi and Peressotti [2021].

Remark 2.3. The operational semantics rely explicitly on the structural equivalence (rule $=_\alpha$) to rename restricted variables via α-equivalence prior to a transition. In paper-based presentations, α-equivalence is often handled implicitly through Barendregt's Variable Convention. In a mechanised setting, however, reasoning over named variables requires freshness conditions, capture-avoiding substitution, and explicit renaming lemmas, introducing substantial proof overhead [Aydemir et al. 2005; Charguéraud 2012]. As discussed in Section 3, this motivates the use of a different binding representation for the syntax of πLL, one designed to reduce explicit α-equivalence reasoning and simplify the treatment of bound variables.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the

$$\frac{}{\vdash P :: \varnothing,\ x:1}\ 1 \quad \xrightarrow{x[]} \quad \vdash P :: \varnothing$$

$$\frac{\vdash P :: \Gamma, x':A \mid \Delta, x:B}{\vdash x[x'].P :: \Gamma, \Delta, x:A \otimes B}\ \otimes \quad \xrightarrow{x[x']} \quad \vdash P :: \Gamma, x':A \mid \Delta, x:B$$

$$\frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x:\bot}\ \bot \quad \xrightarrow{x()} \quad \vdash P :: \Gamma$$

$$\frac{\vdash P :: \Gamma, x':A, x:B}{\vdash x(x').P :: \Gamma, x:A \parr B}\ \parr \quad \xrightarrow{x(x')} \quad \vdash P :: \Gamma, x':A, x:B$$

$$\frac{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash P' :: \mathcal{G}' \qquad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l} \dfrac{\vdash P' :: \mathcal{G}' \quad \vdash Q :: \mathcal{H}}{\vdash P' \mid Q :: \mathcal{G}' \mid \mathcal{H}}\ \text{MIX}}\ \text{PAR}_1$$

$$\frac{\vdash Q :: \mathcal{H} \xrightarrow{l} \vdash Q' :: \mathcal{H}' \qquad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l} \dfrac{\vdash P :: \mathcal{G} \quad \vdash Q' :: \mathcal{H}'}{\vdash P \mid Q' :: \mathcal{G} \mid \mathcal{H}'}\ \text{MIX}}\ \text{PAR}_2$$

$$\frac{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash P' :: \mathcal{G}' \qquad \vdash Q :: \mathcal{H} \xrightarrow{l'} \vdash Q' :: \mathcal{H}' \qquad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{\dfrac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}}\ \text{MIX} \xrightarrow{l|l'} \dfrac{\vdash P' :: \mathcal{G}' \quad \vdash Q' :: \mathcal{H}'}{\vdash P' \mid Q' :: \mathcal{G}' \mid \mathcal{H}'}\ \text{MIX}}\ \text{SYN}$$

$$\frac{P =_\alpha Q \qquad \vdash Q :: \mathcal{G} \xrightarrow{l} \vdash Q' :: \mathcal{G}'}{\vdash P :: \mathcal{G} \xrightarrow{l} \vdash Q' :: \mathcal{G}'}\ =_\alpha \qquad \frac{\dfrac{\vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot \quad \xrightarrow{x[]\mid y()} \quad \vdash P' :: \mathcal{G} \mid \Gamma}{\vdash P :: \mathcal{G} \mid x:1 \mid \Gamma, y:\bot}\ 1\bot \quad \xrightarrow{\tau} \quad \vdash P' :: \mathcal{G} \mid \Gamma}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma}\ \text{CUT}$$

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp \quad \xrightarrow{x[x']\mid y(y')} \quad \vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \parr B^\perp}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \text{CUT} \quad \xrightarrow{\tau} \quad \dfrac{\dfrac{\vdash P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\vdash \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, x:B \mid \Delta, \Xi, y:B^\perp}\ \text{CUT}}{\vdash \nu xy\, \nu x'y'\, P' :: \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \text{CUT}}\ \otimes\parr$$

$$\frac{\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp \xrightarrow{l} \vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp \qquad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\dfrac{\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp}{\vdash \nu xy\, P :: \mathcal{G} \mid \Gamma, \Delta}\ \text{CUT} \xrightarrow{l} \dfrac{\vdash P' :: \mathcal{G}' \mid \Gamma', x:A \mid \Delta', y:A^\perp}{\vdash \nu xy\, P' :: \mathcal{G}' \mid \Gamma', \Delta'}\ \text{CUT}}\ \text{RES}$$

Fig. 2. πMLL, transition rules for derivations.

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0 \quad \dfrac{}{\vdash x[].\mathbf{0} :: x : 1}\,1 \qquad \dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0 \quad \dfrac{}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \bot}\,\bot}{\dfrac{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \bot}{\vdash \nu xz\,(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1}\,\mathrm{CUT}}\,\mathrm{MIX}$$

$\downarrow_{\tau}$ (by rules SYN and $1\bot$, and the transition axioms for rules 1 and $\bot$)

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0 \quad \dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0 \quad \dfrac{}{\vdash y[].\mathbf{0} :: y : 1}\,1}{\vdash \mathbf{0} \mid y[].\mathbf{0} :: \varnothing \mid y : 1}\,\mathrm{MIX}}$$

$\downarrow_{y[]}$ (by rule PAR_2, the transition axiom for rule 1)

$$\dfrac{\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0 \quad \dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0}{\vdash \mathbf{0} \mid \mathbf{0} :: \varnothing \mid \varnothing}\,\mathrm{MIX}$$

$\downarrow_{\equiv}$ (by process congruence, and the monoidal structure of typing environments)

$$\dfrac{}{\vdash \mathbf{0} :: \varnothing}\,\mathrm{MIX}_0$$

Fig. 3. An execution for the derivation in Example 2.2.

set of transition labels, defined as $Lbl = \{\tau, l, (l|l') \mid l, l' \in Act, \mathrm{i}(l) \cap \mathrm{i}(l') = \emptyset\}$. The restriction imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

Example 2.4. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (1, $\bot$, $\otimes$, $\parr$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l|l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\bot$ and $\otimes\parr$ simplify the term into an internal τ-transition, modeling the synchronisation of the session.

2.6 Process Semantics and Erasure

While the typing derivations of πLL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π-calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti 2021] and define an erasure function, $proc : Der \to Proc$, which strips away the typing information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed πLL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

Example 2.5. Consider the well-typed process P formed by adding a third parallel component to the derivation of Example 2.2 before the cut (A derivation of which can be seen on Figure 4):

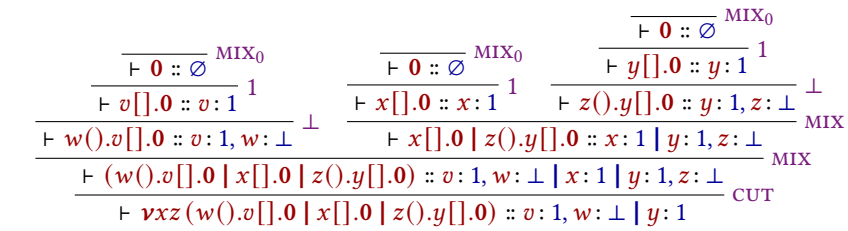

Fig. 4. A derivation for the process $\nu xz\, w().y[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}$

$$P = \nu xz\,(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$$

The component containing w and v is independent from the component containing x, z, and y. This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0} \qquad P \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{w()} \xrightarrow{v[]} \mathbf{0}$$

$$P \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{\tau} \xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} \qquad P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0}$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated in [Montesi and Peressotti 2021], this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times Lbl$ be endofunctors representing the states and labeled transitions, respectively. The erasure function *proc* acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields an Erasure theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

Theorem 2.6 (Erasure). *The process LTS (lts_p) enjoys erasure with respect to the derivation LTS (lts_d). Concretely, for any valid typing derivation $\mathcal{D}$ and label $l \in Lbl$:*

(1) ***Soundness of Erasure:*** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$.*

(2) ***Completeness of Erasure:*** *If $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.*

Corollary 2.7 (Typability Preservation). *If P is well-typed and $P \xrightarrow{l} P'$, then P' is well-typed.*

This theorem allows us to define the operational target for πLL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

Remark 2.8. The untyped LTS for processes relies heavily on explicit side conditions regarding free and introduced names to prevent the collision of bound variables (e.g., ensuring $i(l) \cap i(l') = \emptyset$). However, when operating at the level of typing derivations, these side conditions become inherently redundant. Because πLL enforces linear resource management, the structural rules of the typed LTS dictate that hyperenvironments can only be merged when their domains are strictly disjoint. Consequently, any well-typed derivation inherently satisfies the name-distinctness and freshness

$$x[x'].P \xrightarrow{x[x']} P \qquad x(x').P \xrightarrow{x(x')} P \qquad x[].P \xrightarrow{x[]} P \qquad x().P \xrightarrow{x()} P$$

$$\frac{P \xrightarrow{l} P' \quad \mathrm{i}(l) \cap \mathrm{f}(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q}\ \mathrm{PAR}_1 \qquad \frac{Q \xrightarrow{l} Q' \quad \mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'}\ \mathrm{PAR}_2$$

$$\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad \mathrm{i}(l|l') \cap \mathrm{f}(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l|l'} P' \mid Q'}\ \mathrm{SYN} \qquad \frac{P \xrightarrow{x[x']||y(y')} P'}{\nu xy P \xrightarrow{\tau} \nu xy\, \nu x'y'\, P'}\ \otimes\!\!\parr$$

$$\frac{P \xrightarrow{x[]||y()} P'}{\nu xy P \xrightarrow{\tau} P'}\ 1\bot \qquad \frac{P \xrightarrow{l} P' \quad x, y \notin \mathrm{f}(l) \cup \mathrm{i}(l)}{\nu xy P \xrightarrow{l} \nu xy P'}\ \mathrm{RES} \qquad \frac{P =_\alpha Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'}\ =_\alpha$$

$$\mathrm{i}(l|l') = \mathrm{i}(l) \cup \mathrm{i}(l') \qquad \mathrm{f}(l|l') = \mathrm{f}(l) \cup \mathrm{f}(l')$$
$$\mathrm{i}(\tau) = \emptyset \qquad \mathrm{f}(\tau) = \emptyset$$
$$\mathrm{i}(x[]) = \mathrm{i}(x()) = \emptyset \qquad \mathrm{f}(x[]) = \mathrm{f}(x()) = \{x\}$$
$$\mathrm{i}(x[x']) = \mathrm{i}(x(x')) = \{x'\} \qquad \mathrm{f}(x[x']) = \mathrm{f}(x(x')) = \{x\}$$

Fig. 5. πMLL, transition rules for processes.

requirements of the underlying untyped calculus, shifting the burden of safety from dynamic transition checks to static structural typing.

Proposition 2.9. *The operational semantics of πLL processes strictly respects the introduction of names via transition labels. Formally, if $P \xrightarrow{l} P'$, then $\mathrm{i}(l) \cap \mathrm{f}(P) = \emptyset$, and If P is well-typed, then $\mathrm{i}(l) = \mathrm{f}(P') \setminus \mathrm{f}(P)$ (TODO: Evaluate relevance - delete depending on if this property is mentioend in later proofs).*

Remark 2.10. Proposition 2.9 is vital for mechanising the metatheory of πLL. Because proof assistants tools require explicit tracking of fresh and bound variable to prevent accidental capture, this proposition acts as the formal bridge between the dynamic semantics and the static typing. It guarantees that a well-typed process evolves predictably, ensuring that it does not spontaneously generate resources during a transition. (TODO: same as Proposition 2.9)

2.7 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In πLL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow Montesi and Peressotti [2021] and define a function $env : Der \to Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, Montesi and Peressotti [2021] derive a LTS specifically for typing environments (lts_e), shown in Figure 6.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x : \bot$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalised by the τ-reflexivity axiom, $\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ-action.

The requirement that a process can only perform actions permitted by its typing environment is known as **Session Fidelity**. Just as the *proc* function established operational correspondence for processes, the *env* function establishes a correspondence for environments.

While the rules in Figure 6 define how environments can transition in isolation, the execution of a well-typed program requires the process and its environment to step in unison. This co-dependent relationship has processes only perform actions permitted by its typing environment, while the

$$x : 1 \xrightarrow{x[]} \varnothing \qquad \Gamma, x : \bot \xrightarrow{x()} \Gamma \qquad \Gamma, \Delta, x : A \otimes B \xrightarrow{x[x']} \Gamma, x' : A \mid \Delta, x : B \qquad \Gamma, x : A \parr B \xrightarrow{x(x')} \Gamma, x' : A, x : B$$

$$\frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G}' \mid \mathcal{H}}\ \text{PAR}_1 \qquad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G} \mid \mathcal{H}'}\ \text{PAR}_2 \qquad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l|l'} \mathcal{G}' \mid \mathcal{H}'}\ \text{SYN}$$

$$\frac{\mathcal{G} \mid \Gamma, x : A^\perp \mid \Delta, y : A \xrightarrow{l} \mathcal{G}' \mid \Gamma', x : A^\perp \mid \Delta', y : A}{\mathcal{G} \mid \Gamma, \Delta \xrightarrow{l} \mathcal{G}' \mid \Gamma', \Delta'}\ \text{RES} \qquad \frac{\mathcal{G} \mid x : 1 \mid \Gamma, y : \bot \xrightarrow{x[]|y()} \mathcal{G} \mid \Gamma}{\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma}\ 1\bot$$

$$\frac{\mathcal{G} \mid \Gamma, \Delta, x : A \otimes B \mid \Xi, y : A^\perp \parr B^\perp \xrightarrow{x[x']|y(y')} \mathcal{G} \mid \Gamma, x : B \mid \Delta, x' : A \mid \Xi, y : B^\perp, y' : A^\perp}{\mathcal{G} \mid \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} \mid \Gamma, \Delta, \Xi}\ \otimes\parr$$

Fig. 6. πMLL, transition rules for typing environments.

environment dynamically mirrors the physical control flow of the process, and is formally captured by the property of **Session Fidelity**. Just as the *proc* function established operational correspondence for the untyped calculus, the *env* function establishes this exact behavioral correspondence for the typing contexts.

Theorem 2.11 (Session Fidelity). *Let $\mathcal{D}$ be a valid typing derivation such that $env(\mathcal{D}) = \mathcal{G}$ and $proc(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition*
$\mathcal{G} \xrightarrow{l} \mathcal{G}'$,
and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.

To illustrate Session Fidelity in action, we can observe how the environment is constrained to follow the exact execution traces of the process it types.

Example 2.12 (Session Fidelity in Execution). Recall the process derivation from Example 2.5, typed by the environment $\mathcal{G} = v : 1, w : \bot \mid y : 1$. The process begins by performing an observable input action on w. By Session Fidelity, the environment must faithfully mimic this action, consuming the corresponding type:

$$v : 1, w : \bot \mid y : 1 \xrightarrow{w()} v : 1 \mid y : 1$$

The resulting derivative of the process is now typed by this new environment. At this intermediate stage, if we were to view the environment LTS in isolation, the pure semantics would theoretically allow an output on y to fire immediately, as $y : 1$ is an available, unblocked resource.

However, Session Fidelity dictates a strict co-dependence. The environment merely mirrors the process, and the underlying process term strictly guards the $y[]$ action behind a prefix, requiring a prior τ-synchronisation. Therefore, the environment must wait. When the process finally resolves its internal communication, the environment mimics it via the τ-reflexivity axiom, remaining structurally unchanged:

$$v : 1 \mid y : 1 \xrightarrow{\tau} v : 1 \mid y : 1$$

Only after this synchronisation unblocks the continuation can the process perform the $y[]$ action, which the environment once again mirrors by consuming the final resource:

$$v : 1 \mid y : 1 \xrightarrow{y[]} v : 1$$

Remark 2.13 (The Mechanisation Gap). Session Fidelity is the ultimate touchstone for validating the safety of a session-typed language. However, verifying this property in a proof assistant exposes

a stark gap between paper mathematics and mechanised logic. On paper, the invariant that an environment *dynamically mirrors* a process is highly intuitive, and transitions are easily massaged using structural equivalences and implicit variable conventions. In a machine, however, formally proving that types are consumed correctly requires a rigorous, algorithmic representation of bound variables and explicit freshness. The theoretical elegance of the environment LTS cannot be mechanised without first establishing a concrete architectural foundation for these variables, which dictates the design choices discussed in the following chapters.

2.8 Managing Bound Variables

Having established the operational semantics of πLL and the ultimate goal of verifying Session Fidelity, we must address the treatment of bound variables and freshness, a central technical challenge that underpins the entire formalisation.

As we have seen, many constructs in πLL introduce scoped identifiers. Restriction binds pairs of session endpoints, input constructs bind received channel names, polymorphic communication binds type variables, and duplication constructs introduce fresh channel endpoints. These bindings appear throughout both the process syntax and the labelled transition system.

The concrete choice of bound and introduced names is semantically irrelevant provided the binding structure is preserved. For example, renaming a restricted channel does not change the behaviour of a process. However, reasoning formally about substitutions, transitions, and typing derivations requires these names, scopes, and freshness conditions to be tracked explicitly.

This principle is formalised as *α-equivalence*, which identifies terms that differ only in the names of their bound variables. For instance, the following two processes are α-equivalent:

$$\nu ab\,(a[].\mathbf{0} \mid b().x[].\mathbf{0}) =_\alpha \nu zw\,(z[].\mathbf{0} \mid w().x[].\mathbf{0})$$

The operational semantics of πLL relies on α-equivalence using rule $=_\alpha$ to rename bound variables on the fly, ensuring that restricted scopes and input prefixes do not accidentally capture free names as processes evolve.

Substitution raises this exact issue. Consider the substitution where the free name x is replaced by z in the right-hand process from the previous example:

$$\nu zw\,(z[].\mathbf{0} \mid w().x[].\mathbf{0})$$

Here z is already bound by the restriction, so naively replacing x with z would cause the restriction to capture the newly inserted z. Before the substitution can proceed, and because restriction binds session endpoints as pairs, both the bound z and w must be α-renamed to fresh names, such as to a and b. Similarly, any binder within the process that would capture the substituted name must also be renamed before substitution continues.

In paper-based presentations this is commonly handled by *Barendregt's variable convention*, which assumes bound variables are always chosen distinct from free variables and one another whenever convenient. This makes proofs readable but remains an informal meta-level assumption.

A proof assistant requires every freshness condition and renaming step to be represented and verified explicitly [Aydemir et al. 2005]. A direct mechanisation using named variables therefore introduces substantial proof overhead for capture avoidance and α-equivalence reasoning. To avoid this, the formalisation adopts a *locally nameless* representation of binding, which eliminates α-equivalence as a proof obligation by construction. This is the subject of the following chapter.

3 Binding Architecture

This chapter develops the locally nameless binding architecture used throughout the formalisation. We begin by stating the design goals that motivate the approach (Section 3.1), then introduce the

locally nameless representation and its core operations (Sections 3.2 and 3.3), and conclude with co-finite quantification (Section 3.4), which governs how inductive definitions interact with binders.

3.1 Design Goals and the Approach

This chapter introduces the binding representation used throughout the formalisation. The representation is motivated by three primary design goals:

(1) **Eliminate α-equivalence overhead.** α-equivalent terms should be represented identically, reducing α-equivalence reasoning to definitional equality within the proof assistant.
(2) **Preserve human readability.** Terms should remain explicit and concrete, keeping substitution and freshness reasoning close to their paper-based presentation.
(3) **Avoid informal meta-level conventions.** The representation should not rely on Barendregt's convention or unverified freshness assumptions; capture avoidance and freshness conditions should instead be enforced by construction.

Although both channels and type variables involve binding, they exhibit fundamentally different operational behaviour. Session channels participate in dynamic communication and substitution across concurrent processes, while polymorphic type variables remain statically scoped within typing derivations. The formalisation therefore adopts different binding strategies for these two classes of identifiers while maintaining a common underlying binding architecture.

To achieve these goals, we adopt a locally nameless binding architecture for the calculus [Charguéraud 2012], separating free identifiers from bound occurrences. Bound variables are represented using indices, causing α-equivalent terms to coincide structurally, while free variables remain explicit and readable.

This separation simplifies substitution by restricting it to free variables while leaving bound indices untouched, thereby avoiding accidental capture by construction. As a result, many proofs that traditionally require substantial renaming infrastructure reduce instead to structural recursion and index manipulations.

The remainder of this chapter develops the theoretical machinery underlying this representation.

3.2 The Locally Nameless Representation

The locally nameless representation, as presented by Charguéraud [2012], separates variables into two disjoint syntactic categories: free names and bound indices. This combines the readability of named syntax for free variables with a structural representation of binding.

(1) **Bound variables** are represented using *De Bruijn indices* [de Bruijn 1972], where each index denotes the distance to its corresponding binder. Since the representation is purely positional, α-equivalent terms become structurally identical.
(2) **Free variables** retain explicit names, preserving the readability of open terms and allowing standard name-based reasoning about environments and transition labels.

By structurally separating these two concepts, the locally nameless approach simplifies standard operations such as capture-avoiding substitution. Since free variables and bound variables inhabit distinct syntactic categories, substitution on free variables is defined to not interact with bound indices. We apply this theory to both the structural and operational layers of πLL.

LN Syntax for Types. In the original presentation of πLL, polymorphic types rely on standard named variables. Under the locally nameless architecture, we expand the syntax of type variables to explicitly distinguish between free and bound occurrences. The grammar for types is updated as

follows:

$$\begin{array}{rcll} T & ::= & \mathrm{free}(x) \mid \mathrm{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\ A, B & ::= & \cdots \mid \exists.A \mid \forall.A \mid X \mid X^{\perp} & (\text{where } X \in T) \end{array}$$

Here, a free type variable carries a unique identifier x, while a bound type variable carries a De Bruijn index k. For example, the polymorphic type $\forall X.\exists Y.(X \parr Y)$ is structurally represented without names using the new syntax as $\forall.\exists.(\mathrm{bound}(1) \parr \mathrm{bound}(0))$, where 0 points to the inner existential binder and 1 points to the outer universal binder.

LN Syntax for Processes. Similarly, the standard process syntax displayed in Section 2.2 is updated to reflect the locally nameless separation of channel endpoints. A channel is split into its bound and free counterparts, where bound channel names are represented by indices, while free channels remain explicit identifiers.

Naively removing the bound names from the process syntax would turn binding operations such as $x[y].P$ and $x(y).P$ into $x[].P$ and $x().P$ making them syntactically indistinguishable from their non-binding unit counterparts. To combat this we annotate anonymous binders using placeholder symbols. We use $\bullet$ for a bound process name, and $\diamond$ for a bound type variable.

$$\begin{array}{rcll} \mathit{Channel} & ::= & \mathrm{free}(x) \mid \mathrm{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\ P, Q & ::= & \cdots \mid x[\bullet].P \mid x(\bullet).P \mid \nu(\bullet,\bullet).\,P \mid x(\diamond).P & (\text{where } x \in \mathit{Channel}) \end{array}$$

While the visual anonymity of these constructors is evident from the grammar, their mechanical implementation relies on precise index management. For a multi-name binder like restriction, $\nu(\bullet,\bullet)\,P$, the two dual endpoints are simultaneously mapped to distinct indices within the continuation of P, namely 0 and 1. For communication prefixes like $x(\bullet).P$, the payload is implicitly bound as index 0 within P, while the explicit free channel x is deliberately preserved as the subject. By maintaining explicit free names strictly for these outward-facing communications, the transition labels and operational semantics of the calculus remain completely unaffected by the architectural shift.

While the locally nameless architecture elegantly resolves α-equivalence, moving between abstracted and instantiated forms requires specialised operations. To execute a substitution or evaluate a process across a binder, we must formally define how to open and close these terms.

3.3 Opening and Closing Binders

Because the locally nameless representation separates free and bound variables, standard substitution cannot be applied uniformly under binders. Instead, moving between bound and free representations is handled by two dual operations, namely *opening* and *closing* [Charguéraud 2012].

Opening (Instantiation). Opening is the operation of traversing into a binding construct and replacing the exposed bound index with a concrete free name. Formally, opening a process P at depth k with a free name x is denoted $\{k \mapsto x\}P$.

The operation proceeds via structural recursion. A depth counter, initially set to 0, tracks the number of binders traversed. As the recursion descends through the syntax tree, the counter increments when passing a new binder. Consider being at depth k, then traversing a single-name binder like $x(\bullet).P$ increments the depth to $k+1$, while traversing a double-name binder like $\nu(\bullet,\bullet)\,P$ would increment it to $k+2$.

When the recursion reaches a bound variable $\mathrm{bound}(n)$, the index n is compared against the current depth k. If $n = k$, the variable refers to the binder currently being opened and is replaced by the free name $\mathrm{free}(x)$. Otherwise, the index refers to a different binder and is left unchanged.

Crucially, when the opening operation encounters an existing free variable $\text{free}(y)$, it simply ignores it. This guarantees that opening is entirely capture-avoiding by construction. In the operational semantics, when a single binder like an input prefix $z(\bullet).P$ receives a channel x, the continuation P is opened as $\{0 \mapsto x\}P$. For a restriction $\boldsymbol{\nu}(\bullet, \bullet)\, P$, the dual endpoints x and y are opened sequentially as $\{0 \mapsto x\}(\{1 \mapsto y\}P)$.

Closing (Abstraction). Closing is the exact inverse of opening. It is the operation of taking an open term and abstracting a specific free name x into a bound index at depth k, denoted $\{k \leftarrow\!\!\shortmid x\}P$. While recursing, bound indices are ignored as they cannot contain free names. When the recursion encounters a free term like $\text{free}(y)$, it checked if $y = x$. If they match, the free name is replaced by $\text{bound}(k)$.

This operation is primarily used when constructing new binders, such as moving a free channel into a restricted scope. To close multiple free names simultaneously, such as abstracting x and y into a restriction, the operations are applied sequentially, by first closing y at depth $k + 1$, followed by closing x at depth k.

Locally Closed Terms. The separation of names and indices introduces a new syntactic anomaly, namely that a term can contain a *dangling* index that does not point to any enclosing binder. For example, the process term $\text{bound}(0)[].\mathbf{0}$ is syntactically valid, matching the syntax for an empty send, but mathematically it is meaningless because the index 0 has no corresponding binder.

To filter out these meaningless terms, the locally nameless architecture introduces the predicate of being *locally closed*, denoted $lc(P)$. A term is locally closed if and only if all of its De Bruijn indices are properly bound by an enclosing constructor.

The locally closed predicate excludes terms containing dangling indices and therefore characterises the well-formed syntax of the calculus. Establishing this predicate is not only a static validation step, but rather it forms a key invariant throughout the mechanisation, which guarantees the soundness of the calculus. Generally, any subsequent definition, such as variable substitution, structural congruence, or operational semantics, must be mathematically proven to *preserve* the locally closed property. In practice, this is achieved through the *regularity of typing*, which requires proving that all valid typing derivations inherently produce locally closed terms. Consequently, operations proven to preserve typing will naturally carry this invariant as a structural guarantee.

3.4 Co-finite Quantification

With the syntax partitioned into names and indices, and the mechanics of opening and closing established, the final theoretical hurdle is defining how inductive relations, such as typing judgements and labelled transitions, operate across binders.

In paper-based proofs, when a rule requires opening a binder, Barendregt's variable convention allows the author to simply state: *pick a fresh name x*. In a proof assistant, this informal generation of a fresh name must be encoded into the constructors of the inductive relations. Two naive approaches to bind this fresh name, both of which introduce severe mechanical flaws [Aydemir et al. 2008; Charguéraud 2012]:

(1) **Existential Quantification ($\exists x \notin \text{f}(P)$):** The rule requires the property to hold for at least one fresh name. While this makes proof construction straightforward, it yields weak induction hypotheses, since induction only provides the property for one particular fresh name chosen during the original derivation.
(2) **Universal Quantification ($\forall x$):** The rule requires the property to hold for every name. This provides a strong induction hypothesis, but is too restrictive for proof construction,

since the property cannot generally be shown for names that already occur free in the surrounding context.

To solve this dilemma, the locally nameless architecture employs **co-finite quantification** [Aydemir et al. 2008; Charguéraud 2012]. Rather than quantifying over one name or all names, the inductive rules quantifies universally over all names *except* those not in an arbitrary, finite set L. Formally, a rule requiring a fresh name uses the premise: $\forall x \notin L$.

By deferring the exact choice of the forbidden set L to the user at the time of proof, co-finite quantification achieves the best of both worlds. When constructing a proof tree, the user can instantiate L as the set of all free variables currently in the environment, process, and transition labels ($\mathrm{f}(P) \cup \mathrm{f}(\Gamma) \cup \dots$). This ensures the premise is trivially satisfiable. Conversely, when performing structural induction, the co-finite universal quantifier guarantees that the induction hypothesis applies to *any* fresh name the user could possibly need, provided they pick one outside the finite set L.

Application to πLL Types. Although type variables are represented using the locally nameless separation of free and bound constructors, their metatheory is handled using depth-indexed reasoning rather than cofinite quantification. Because type variables are static and do not dynamically communicate across parallel scopes like channels do, managing freshness sets L for them introduces unnecessary proof overhead.

Instead, the system evaluates polymorphic typing rules using a purely depth-indexed approach, powered by a standard De Bruijn shifting operator. We define the shifting of a type A by a distance d at a cutoff depth c, denoted mathematically as $\uparrow_d^c A$. Using this infrastructure, consider the typing rule for the universal type input ($\forall.B$):

$$\frac{n+1 \vdash P :: \Gamma^{\uparrow t}, x : B}{n \vdash x(\diamond).P :: \uparrow_0^1 \Gamma, x : \forall.B}\ \forall_{\mathrm{DB}}$$

Rather than opening the binder with a fresh explicit name, the typing judgment simply increments a depth counter $n+1$, legally bringing index 0 into scope. To maintain correct index distances, the typing environment Γ is explicitly shifted by a distance of 1 at cutoff 0 ($\uparrow_0^1 \Gamma$), incrementing all free type indices within the context to prevent accidental capture.

Application to πLL Channels. This co-finite quantification strategy fundamentally reshapes the inductive definitions for process channels. For example, consider the typing rule that introduces the prefix $x(\bullet).P$. Under a paper-based presentation, the premise implicitly requires a fresh session name y. Under our mechanised architecture, the rule uses co-finite quantification and the opening operation:

$$\frac{\forall y \notin L, \quad n \vdash \{0 \mapsto y\}P :: \Gamma, x : B, y : A}{n \vdash x(\bullet).P :: \Gamma, x : A \parr B}\ \parr_{\mathrm{LN}}$$

Here, the rule states that the process is well-typed if, for any fresh channel variable y outside some finite set of forbidden variables L, the opened process P is well-typed under the environment extended with the opened payload. By utilizing this mixed evaluation strategy, co-finite quantification for dynamic channels, and depth-indexed evaluation for static types, the architecture minimizes bureaucratic renaming overhead while maintaining a unified underlying syntax.

With this mathematical foundation established, the conceptual theory of πLL is fully prepared for implementation. The following chapter details how this locally nameless architecture is concretely mechanised in the Lean 4 proof assistant.

4 Formalisation

This section describes the mechanisation of πLL in Lean 4, bridging the theoretical foundations developed in Sections 2 and 3 and their concrete implementation. The formalisation is organised into three top-level namespaces: `PiLL.Model`, which defines the static structure of the calculus, `PiLL.Semantics`, which defines its operational behaviour, and `PiLL.SessionFidelity`, which establishes the metatheoretic results connecting the two. Each component is presented in dependency order, highlighting design decisions and divergences from Montesi and Peressotti [2021].

4.1 Model

The `Model` namespace covers the full πLL calculus and is structured around the layers introduced in Section 2. `Base.lean` provides shared syntactic infrastructure used across the model. Session types are defined in `STypes/`, processes in `Processes/`, and typing contexts in `Environments/` and `HyperEnvironment/`. Typing judgements and their properties are collected in `Judgement/`, which includes inversion lemmas, linearity properties, substitution, and the co-finite freshness infrastructure described in Section 3.4.

4.1.1 Session Types. Session types are defined in `STypes/Basic.lean` as the inductive datatype `Types`. Following the locally nameless syntax established in Section 3.2, type variables are isolated into a separate inductive type, `TVar`, which explicitly distinguishes between free and bound occurrences:

```
inductive TVar : Type
  | free  (x : FTVar)
  | bound (x : BTVar)
```

Here, `FTVar` represents a free type variable identified by a natural number, while `BTVar` represents a bound type variable encoded as a De Bruijn index.

The remaining constructors directly mirror the theoretical grammar of the calculus. In addition, the formalisation introduces constructors for uninterpreted atomic base types (`atom` and `atomDual`), which provide the base cases for the inductive structure of the syntax. Table 3 illustrates the correspondence between the paper syntax, the locally nameless representation, and the Lean implementation.

Table 3. Comparison of standard, locally nameless, and Lean representations for πLL types.

Standard	Locally Nameless	Lean Constructor	Binds
$A \otimes B$	$A \otimes B$	`| tensor (A B : Types)`	None
1	1	`| one`	None
$\forall X.A$	$\forall.A$	`| forall_ (A : Types)`	1 type
$\exists X.A$	$\exists.A$	`| exists_ (A : Types)`	1 type
X	X	`| var (v : TVar)`	None

Notably, the polymorphic binders $\forall.A$ and $\exists.A$ reflect the depth-indexed approach detailed in Section 3.4. The constructors keep the bound De Bruijn index implicit, taking only the continuation body `A` as an argument (an underscore "_" has been appended to the constructor names to prevent namespace collisions with Lean's built-in `forall` and `exists` keywords).

Duality. `Types.dual` implements duality as a structurally recursive function, mapping each constructor to its dual. For example:

```
def Types.dual : Types -> Types
  | .tensor A B  => .parr    (dual A) (dual B)
  | .forall_ A   => .exists_ (dual A)
  | .one         => .bot
  | ...
```

The fundamental property that duality is an involution, $(A^\perp)^\perp = A$, is proved in `STypes/Lemmas.lean` by structural induction.

The formalisation additionally establishes that duality preserves local closure and commutes with both shifting and substitution:

$$A.\mathtt{lc} \iff A^\perp.\mathtt{lc} \qquad \uparrow_d^c (A^\perp) = (\uparrow_d^c A)^\perp \qquad B^\perp\{A/k\} = (B\{A/k\})^\perp$$

Local closure. `STypes/Properties.lean` defines local closure for types as a depth-indexed predicate `Types.lc (k : Nat) : Types → Prop`, where the depth parameter k tracks how many binders the recursion has traversed.

At the leaves of the syntax tree, local closure is evaluated by the helper predicate `TVar.lc`. Because free variables are explicit they do not contain any indices, as such they are inherently locally closed at any depth. Conversely, a bound variable is only locally closed if its De Bruijn index i is strictly less than the current depth k, ensuring it points to an enclosing scope:

```
def TVar.lc : Nat -> TVar -> Prop
  | _, .free _   => True
  | k, .bound i  => i < k
```

The primary `Types.lc` predicate is then defined by structural recursion. For non-binding constructors, the depth parameter distributes across the sub-terms. When a variable is encountered, the predicate delegates the check to `TVar.lc`. When descending into the body of a polymorphic binder, the depth counter increments by one, bringing the newly introduced index safely into scope:

```
def Types.lc : Nat -> Types -> Prop
  | _, .one         => True
  | k, .var v       => TVar.lc k v
  | k, .tensor A B  => A.lc k ∧ B.lc k
  | k, .forall_ A   => A.lc (k + 1)
  | k, .exists_ A   => A.lc (k + 1)
  | ...
```

Finally, a type is defined as *closed* if it contains no dangling indices at the top level, meaning it is locally closed at depth 0. This is registered as the abbreviation `Types.lc_0`.

Shifting. `STypes/Shift.lean` defines shifting following the standard De Bruijn shifting operation introduced in Section 3.4. The operation traverses the syntax tree structurally using a cutoff depth d and a shift amount c.

For standard connectives, shifting distributes recursively across subterms, while base constructors such as `one` and `bot` remain unchanged. When traversing a polymorphic binder like `forall_`, the cutoff depth increments by one to account for the newly entered scope:

```
def Types.shift (d c : Nat) : Types -> Types
  | .var v       => .var (v.shift d c)
  | .tensor A B  => .tensor (A.shift d c) (B.shift d c)
```

```
  | .forall_ A   => .forall_ (A.shift (d + 1) c)
  | t           => t
```

At the variable level, free variables remain unchanged. For bound indices, the function compares the De Bruijn index i against the cutoff depth d. Indices satisfying $i < d$ are locally bound within the traversed scope and therefore remain unchanged. Otherwise, the index is incremented by c to preserve its reference to the surrounding environment:

```
def TVar.shift (d c : Nat) : TVar -> TVar
  | .bound i => if i < d then .bound i else .bound (i + c)
  | .free n  => .free n
```

Substitution. `STypes/Subst.lean` defines substitution as the eager substitution of A for a target De Bruijn index k within a host type B, denoted as $B\{A//k\}$.

At the variable level, the function evaluates the bound De Bruijn index i against the target depth k. If $i = k$, the target has been reached, and the variable is replaced by the instantiated type A. If $i < k$, the variable is bound by an inner scope and is unaffected. Finally, if $i > k$, the index refers to an outer binder beyond the substituted variable. Since substitution removes the binder corresponding to k, these indices are decremented by one to preserve their relative distance to the surrounding environment.

```
def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.bound i) =>
      if i == k then A
      else if i > k then .var (.bound (i - 1))
      else .var (.bound i)
```

This operation is lifted to the `Types` level via structural recursion. For standard connectives, the substitution simply distributes across the sub-terms. However, when descending into a polymorphic binder such as `forall_`, two critical protective adjustments must occur. First, the target depth k increments by one ($k + 1$) to account for crossing the new binder. Second, the payload type A is explicitly shifted (`shift 0 1 A`) to ensure that any free variables residing inside A are not accidentally captured by this newly entered scope:

```
  | .forall_ B => .forall_ (B.subst (shift 0 1 A) (k + 1))
```

Notation. `STypes/Notation.lean` defines notation following the mathematical conventions established in Section 2.3 and extended in LN Syntax for Types. The only noteworthy divergence is that the exponentials $!A$ and $?A$ are written $!!A$ and $??A$ respectively, to avoid clashing with Lean's built-in ! (boolean negation) and ? (syntactic hole) operators.

The full inductive definition of `Types` and all lemmas from `STypes/Lemmas.lean` are listed in **??**.

4.1.2 Processes. Processes are defined in `Processes/Basic.lean` as the inductive type `Proc`. Following the established locally nameless architecture, channel references are partitioned into a dedicated `Channel` inductive. Explicit free channels are instantiated as concrete natural numbers (`FPName`). While the raw process syntax permits arbitrary reuse of these numbers, using `Nat` allows the formalisation to systematically compute strictly fresh names by incrementing the supremum of any finite set of existing names (mechanised in `Processes/Fresh.lean`). Conversely,

bound channels are represented using De Bruijn indices, encoding their relative distance to the corresponding binder.

```
inductive Channel : Type where
  | free  (x : FPName)
  | bound (x : BPName)
```

As discussed in Section 3.2, binding constructs in the process syntax no longer carry explicit payload variables. Instead, bound occurrences are captured purely by their relative position. Comparing the standard paper grammar with the intermediate locally nameless notation and the final constructors of the `inductive Proc` type illustrates this abstraction:

Standard	Locally Nameless	Lean Constructor	Binds
$x[y].P$	$x[\bullet].P$	`| tensor (x : Channel) (P : Proc)`	1 channel
$x(y).P$	$x(\bullet).P$	`| parr (x : Channel) (P : Proc)`	1 channel
$\nu xy\,P$	$\nu(\bullet,\bullet)\,P$	`| cut (P : Proc)`	2 channels
$x(X).P$	$x(\diamond).P$	`| input (x : Channel) (P : Proc)`	1 type
$x[\textsc{dup}](x').P$	$x[\textsc{dup}](\bullet).P$	`| duplicate (x : Channel) (P : Proc)`	1 channel

Notice that `cut` takes only a continuation P with no explicit channel arguments, as both dual endpoints are simultaneously bound as indices 0 and 1 within the scope of P. Similarly, for constructs like `tensor`, `parr`, and `input`, the subject channel x over which the interaction occurs is kept explicit, while the transmitted payload is anonymized into the continuation.

Opening and closing. `Processes/OpenClose.lean` defines opening and closing operations following the Locally Nameless principles established in Section 3.3.

Because binders are represented positionally, the depth parameter k increments according to the number of channels bound by each constructor. For standard communication prefixes like `tensor`, `parr`, and the server `duplicate` operation, exactly one channel is bound, incrementing k by one. The restriction constructor (`cut`) simultaneously binds two dual endpoints, requiring k to increment by two. Notably, the `input` constructor binds a type variable rather than a channel. Consequently, it does not affect the channel depth parameter k.

```
def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
  match P with
  | .tensor x P     => .tensor (x.open u k) (P.open u (k + 1))
  | .parr   x P     => .parr   (x.open u k) (P.open u (k + 1))
  | .cut      P     => .cut                 (P.open u (k + 2))
  | .input  x P     => .input  (x.open u k) (P.open u k)
  | ...
```

At the leaves of the syntax tree, this recursive traversal delegates to the base case defined by `Channel.open`. For bound channels, the function evaluates the De Bruijn index i against the target depth k, replacing it with the instantiated channel v upon an exact match. Explicit free channels remain entirely unaffected:

```
def Channel.open (u v : Channel) (k : Nat) : Channel :=
  match u with
  | bound i => if i == k then v else bound i
  | c => c
```

For restriction, which requires instantiating two endpoints simultaneously, the `HasOpenTwo` typeclass from `Base.lean` is instantiated by opening sequentially: first replacing index $k+1$ with the second endpoint v, then replacing index k with the first endpoint u:

```
instance : HasOpenTwo Proc Channel Channel Nat where
  open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)
```

To complete the bidirectional infrastructure, `Proc.close` and `Channel.close` define the exact inverse operation. Used primarily when abstracting explicit free names into new scopes, `Proc.close` traverses the process tree using a structural recursion identical to `Proc.open`, applying the exact same depth-incrementing rules for single and double binders. When the recursion reaches a leaf node, it delegates to `Channel.close`. If the leaf is a free channel (`FPName`), the function compares it against the target name x, upon a match, the explicit name is *abstracted* into a bound De Bruijn index at the accumulated depth k:

```
def Channel.close (u : Channel) (x : FPName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i
```

Local closure. `Processes/LocalClosure.lean` defines local closure for processes as a two-parameter predicate `Proc.lc (k n : Nat) : Proc → Prop`. Because the calculus features a two-sorted syntax, the predicate must maintain independent depth counters: k tracks the depth of channel binders, while n tracks the depth of type binders. This ensures that when a process embeds a type, such as the payload A in an output operation, the embedded type can be correctly evaluated against the current type depth n during its own local closure check.

The structural recursion evaluates each constructor by incrementing the appropriate depth counter according to the number and sort of binders introduced. Channel binders increase the channel depth k, while type binders increase the type depth n. For example, `tensor` binds a single continuation channel, incrementing the depth to $k+1$, whereas `cut` binds two dual endpoints simultaneously, incrementing the depth to $k+2$. Conversely, `input` binds a type variable, leaving k unchanged while incrementing n to $n+1$. Constructors introducing no binders leave both counters unchanged:

```
def Proc.lc : Nat -> Nat -> Proc -> Prop
  | _, _, .nil          => True
  | k, n, .one x P      => x.lc k ∧ P.lc k n
  | k, n, .tensor x P   => x.lc k ∧ P.lc (k + 1) n
  | k, n, .cut P        => P.lc (k + 2) n
  | k, n, .input x P    => x.lc k ∧ P.lc k (n + 1)
  | ...
```

A process is considered globally *closed* when it contains no dangling indices of either sort, meaning `Proc.lc 0 0` holds. This is registered as the abbreviation `Proc.lc_0`.

Free names. `Processes/Names.lean` defines `Proc.f : Proc → Finset FPName` to compute the free channel names of a process. The function traverses the syntax tree structurally, collecting all explicit free channel identifiers into a finite set. Because locally nameless binders are represented using anonymous indices, bound channels contribute nothing to this set.

For constructors carrying explicit subject channels, such as `tensor` and `link`, the free names of those channels are combined with the recursively computed free names of their continuations.

Constructors without explicit channel identifiers of their own, such as `cut`, contribute only the free names recursively obtained from their continuations:

```
def Proc.f : Proc -> Finset FPName
  | .cut P      => P.f
  | .par P Q    => P.f ∪ Q.f
  | .link x y   => x.f ∪ y.f
  | .tensor x P => x.f ∪ P.f
  | .one x P    => x.f ∪ P.f
  | .nil        => {}
  | ...
```

Freshness. `Processes/Fresh.lean` establishes freshness to support the generation of globally unique names. Given a finite set L of forbidden names, a strictly fresh name is deterministically computed as $\max(L) + 1$. The formalisation provides two critical lemmas, `exists_one_fresh` and `exists_two_fresh`, which guarantee the generation of one or two distinct names residing entirely outside of L. These lemmas are essential for providing witnesses to satisfy the co-finite quantification premises used throughout the typing and transition rules, particularly for restriction, which requires introducing two fresh, non-colliding endpoints simultaneously.

Name substitution. `Processes/SubstNames.lean` establishes the machinery for name substitution as `Proc.substNames (R T : FPName) : Proc → Proc`, replacing all explicit free occurrences of the target name T with the replacement name R. Because the locally nameless architecture strictly partitions free names and bound indices into disjoint syntactic categories, substituting a free name R only interacts with other explicit free names at the leaves of the syntax tree. Binders, which operate exclusively via relative De Bruijn indices, are fundamentally blind to the concrete name R. Therefore, this operation is intrinsically capture-avoiding by construction, completely eliminating the need for complex freshness constraints or α-renaming during substitution.

Type substitution. `Processes/SubstTypes.lean` establishes the machinery needed for type substitution as `Proc.substTypes (A : Types) (k : Nat) : Proc → Proc`, eagerly substituting A for the De Bruijn index k throughout a process. Because the syntax is two-sorted, the operation delegates to the type-level substitution machinery whenever it encounters an embedded type. For example, within an `output` prefix, `Types.subst` is applied directly to the transmitted type. When traversing an `input` prefix, which binds a type variable, the substitution descends with an incremented depth $k + 1$. Simultaneously, A is explicitly shifted (`A.shift 0 1`) to prevent its free variables from being captured by the newly entered binder, mirroring the pattern used in `Types.subst` from Section 4.1.1:

```
def Proc.substTypes (A : Types) (k : Nat) : Proc -> Proc
  | .input x P => .input x (P.substTypes (A.shift 0 1) (k + 1))
```

A consequence of the locally nameless representation is that no channel shifting operation (e.g. `shiftNames`) is required. In a pure De Bruijn encoding such as that used for `Types` in Section 4.1.1, variables are represented as relative indices and must be shifted when traversing binders to preserve correct binding structure. By contrast, free channels (`FPName`) are represented as global identifiers rather than relative positions, and are therefore unaffected by binder depth. This avoids the need for structural adjustment of channel names during substitution, simplifying the process-level infrastructure.

Structural Properties. `Processes/Lemmas.lean` establishes the fundamental structural properties required by the later typing and operational metatheory. This includes commutation lemmas for interacting operations (e.g., `open_substNames_comm`, showing that opening commutes with name substitution), preservation properties for local closure (e.g., `lc_of_open`), and lemmas relating binders to free-name analysis (e.g., `f_close`, proving that closing removes a free name from the resulting term).

These results provide the core infrastructure required for the typing and transition semantics developed in subsequent chapters.

For completeness, the full inductive definition of `Proc` and all lemmas from `Processes/Lemmas.lean` are listed in **??**.

4.1.3 Environments. Environments are defined in `Environment/Basic.lean` as `Env`, an abbreviation for a list of channel-type pairs:

```
abbrev Elem := (FPName × Types)
abbrev Env  := List Elem
```

The choice of `List` as the underlying representation deserves explanation, as it was not the first approach attempted. An initial implementation used `Finset` to directly model the unordered, duplicate-free nature of the theoretical environment. However, because `Finset` in Lean is implemented as a quotient type over lists, it inherently resists direct structural induction and pattern matching. This led to two distinct hurdles, namely the constant occurrence of dependent elimination failures, and the necessity of lifting functions and proofs reasoning about environments across the quotient equivalence. In the context of a process calculus, where inversion and structural manipulation of the typing context are omnipresent, this quotient infrastructure proved unwieldy. A second attempt using association lists (`AList`) encountered similar difficulties. The final design uses a plain `List` and recovers the unordered structure through an explicit permutation relation, trading quotient reasoning for structural induction, which interacts far more smoothly with Lean's tactic machinery.

Recovering the PCM structure. Merging two environments is implemented as list concatenation, making it a simple structural, but total, function:

```
abbrev Env.merge (Γ Δ : Env) : Env := Γ ++ Δ
infixl:69 "," => Env.merge
```

Because lists do not naturally form a Partial Commutative Monoid (PCM) but are associative by definition, we only need to formally recover commutativity. This is achieved by reasoning up to permutation equivalence, using Lean's built-in `List.Perm` relation, which is instantiated through a custom `HasPerm` typeclass and denoted by the infix relation $\Gamma \sim \Delta$. The essential monoidal laws are established as:

```
lemma Env.merge_comm (Γ Δ : Env) : Γ,Δ ~ Δ,Γ
lemma Env.merge_assoc (Γ Δ Ξ : Env) : Γ,Δ,Ξ = Γ,(Δ,Ξ)
```

Additionally, we use $\varnothing$ as unit for merging and implement identity for merging it on both the left and right side of an environment as well as some useful lemmas for defining a standard representation for example turning $e :: \varnothing$ into $[e]$.

Remark 4.1. The use of $\varnothing$ as unit has necessitated explicit lemmas to rewrite and reduce terms such as $\varnothing, \Gamma$ to Γ. Whereas if $[]$ were used as unit terms like $[], \Gamma$ would reduce by definitional equality, because the `Env.merge` notation is an abbreviation for `List.append`, Lean can see through it and reduce automatically.

The requirement that merging is only valid for disjoint environments, is partiality enforced via explicit predicates rather than a partial function. `Env.names` extracts the domain as a `Finset`, enabling Lean's built-in `Disjoint` typeclass. Internal linearity (no channel typed twice) is enforced by `Env.Nodup` (using List.Nodup as the underlying structure):

```
def Env.names (Γ : Env) : Finset FPName := (Γ.map Prod.fst).toFinset
def Env.disjoint (Γ Δ : Env) : Prop := Disjoint Γ.names Δ.names
def Env.Nodup (Γ : Env) : Prop := (Γ.map Prod.fst).Nodup
```

The interaction between these predicates and merging is captured by `Env.Nodup_merge_iff`, which states that a merged environment is linear if and only if both components are linear and mutually disjoint.

Server usability. The typing rule for server replication requires all dependency channels to carry exponential types. Since Lean inductive constructors cannot explicitly express an arbitrary list of type constraints, this is abstracted into the `Env.serverUsable` predicate. This asserts that every type in a given environment satisfies `Types.isServerUsable`, a type-level predicate checking whether a type takes the form of an exponential request (?B).

```
def Env.serverUsable (Γ : Env) : Prop :=
  ∀p, p ∈ Γ -> p.snd.isServerUsable
prefix:max "?ₑ" => Env.serverUsable
```

Lifted operations. Since environments contain types, all locally nameless operations from Section 4.1.1 are lifted pointwise over the type component of each `Elem`. Local closure, type shifting, name substitution, and type substitution are all defined by mapping over the list, and `Environment/Lemmas.lean` proves that each operation preserves the names, disjointness, nodup, and permutation properties of the environment.

The full definitions and lemmas are listed in **??**.

4.1.4 Hyperenvironments. Hyperenvironments are defined in `HyperEnvironment/Basic.lean` as `HyperEnv`, an abbreviation for a list of environments:

```
abbrev HyperEnv := List Env
```

Merging is again list concatenation, using $\varnothing$ as unit and $|_h$ as merge operator to avoid clashing with Lean's built-in $|$, with the unit and associativity laws holding definitionally, mirroring the environment design. However, recovering the commutative structure required a more extensive permutation relation than the one used for environments.

Deep permutation. For environments, `List.Perm` sufficed, as it allowed arbitrary reordering of elements. For hyperenvironments this is not enough. A hyperenvironment is a list of environments, so not only should reordering of the environments be allowed, but the environments themselves should also be permutation-equivalent.

Example 4.2. Consider the hyperenvironment $[x : A, y : B]$, consisting of a single environment with two channel-type pairs. Under `List.Perm`, this would *not* be equivalent to $[y : B, x : A]$, since the single environment element is treated as an opaque value. Under `HyperEnv.Perm`, the `cons` constructor permits the head environment to be replaced by any permutation-equivalent environment, so $[x : A, y : B] \sim [y : B, x : A]$ holds via `cons` with the environment permutation $x : A, y : B \sim y : B, x : A$.

Standard `List.Perm` is a *shallow* permutation implementation, meaning that it does not capture this. Here each environment is treated as an opaque element and cannot be permute. To address this, a custom `HyperEnv.Perm` inductive is defined:

```
inductive HyperEnv.Perm : HyperEnv -> HyperEnv -> Prop where
  | nil  : Perm [] []
  | cons : {Γ Δ : Env} {𝒢 ℋ : HyperEnv} : Γ ~ Δ -> Perm 𝒢 ℋ -> Perm (Γ :: 𝒢) (Δ :: ℋ)
  | swap : {Γ Δ : Env} {𝒢 : HyperEnv} : Perm (Γ :: Δ :: 𝒢) (Δ :: Γ :: 𝒢)
  | trans {𝒢 ℋ 𝒥 : HyperEnv} : Perm 𝒢 ℋ -> Perm ℋ 𝒥 -> Perm 𝒢 𝒥
```

The `cons` constructor is the key distinction from `List.Perm`. Rather than requiring the head elements to be identical, it requires them to be *environment-permutation-equivalent* ($\Gamma \sim \Delta$). This makes `HyperEnv.Perm` a *deep* permutation implementation that propagates equivalence down to it elements as well as its own ordering. The remaining constructors mirror `List.Perm` directly.

Well-formedness. Hyperenvironments carry two distinct linearity conditions. `HyperEnv.Nodup` differs from `Env.Nodup`, by only requiring that each component environment is internally linear (no channel typed twice within a single environment), while ensuring that merging only occurs for disjoint environments is handled by `HyperEnv.PairwiseDisjoint`, which requires that the component environments are mutually disjoint, which ensures that no channel appears in two distinct components:

```
def HyperEnv.Nodup (𝒢 : HyperEnv) : Prop :=
  ∀ Γ,  Γ ∈ 𝒢 -> Env.Nodup Γ

def HyperEnv.PairwiseDisjoint (𝒢 : HyperEnv) : Prop :=
  List.Pairwise Env.disjoint 𝒢
```

Example 4.3. To illustrate this distinction, consider three candidate hyperenvironments under a naming context where x, y, z are distinct channel names:

- **Well-formed:** $\mathcal{G}_1 = [x : A] \mid_h [y : B, z : C]$ satisfies both predicates. Each component list has no duplicate channels, and their domains share no common names.
- **Fails Nodup:** $\mathcal{G}_2 = [x : A, x : B] \mid_h [y : C]$ satisfies `PairwiseDisjoint` because the two components do not share names, but it violates `Nodup` because the first environment has multiple occurences of x.
- **Fails PairwiseDisjoint:** $\mathcal{G}_3 = [x : A] \mid_h [x : B, y : C]$ satisfies `Nodup` because both component environments are internally valid, but it violates `PairwiseDisjoint` because the channel x crosses the parallel composition boundary and appears in both components.

Together these two predicates enforce the global linearity of a hyperenvironment, namely that every channel appears at most once across all components.

Lifted operations. As with environments, type shifting, name substitution, and type substitution are all lifted pointwise by mapping over the list of component environments. `HyperEnvironment/Lemmas/Basic.lean` proves that each operation preserves names, disjointness, pairwise disjointness, and the deep permutation relation. The preservation of permutation under shifting is a representative example, it proceeds by induction on `HyperEnv.Perm`, using the corresponding environment-level lemma at the `cons` case. The full definitions and lemmas are listed in **??**.

4.1.5 Typing Judgements. The typing relation is defined in `Judgement/Basic.lean` as the inductive predicate `Typing`:

```
inductive Typing : Nat -> Proc -> HyperEnv -> Prop
```

We write $n \vdash P :: \mathcal{G}$ to refer to this construct. The natural number n tracks the current type binder depth, as established in Section 4.1.2. Whenever the derivation descends into a `forall_` constructor, n is incremented and the environment is shifted by one to keep all embedded type indices in scope:

```
| forall_ {Γ : Env} {P : Proc} {x : FPName} {B : Types} {n : Nat} :
    Typing (n + 1) P [x : B :: Γ⁺] ->
    Typing n (#x((⋄)).P) [x : ∀.B :: Γ]
```

The shift Γ^+ (or equivalently $\Gamma \uparrow 0, 1$) applied to the environment ensures that all free type indices in Γ are incremented to account for the newly introduced binder, mirroring the shift applied during type substitution in Section 4.1.1.

Exchange rules. Rather than building commutativity into every constructor, the typing relation includes two explicit structural rules that allow reasoning up to permutation equivalence:

```
| exchange_env   : Typing n P (Γ :: 𝒢) -> Γ ~ Δ -> Typing n P (Δ :: 𝒢)
| exchange_hyper : Typing n P 𝒢 -> 𝒢 ~ ℋ -> Typing n P ℋ
```

`exchange_env` allows replacing a component environment with any permutation-equivalent one, while `exchange_hyper` allows replacing the entire hyperenvironment. Together these recover the unordered character of the theoretical presentation from Section 4.1.3 without requiring a quotient type.

Co-finite quantification. Binding constructors use co-finite quantification as described in Section 3.4. For example, the `cut` constructor introduces two fresh dual endpoints:

```
| cut {𝒢 : HyperEnv} {Γ Δ : Env} {P : Proc} {A : Types} {n : Nat} (L : Finset FPName) :
    ∀x ∉ L, ∀y ∉ L, x ≠ y -> Typing n (P((#x, #y))) (𝒢 |ₕ [x : A :: Γ] |ₕ [y : A^⊥ :: Δ]) ->
    Typing n (ν((•, •)) P) (𝒢 |ₕ [Γ, Δ])
```

The premise holds for any two distinct names x, y outside the finite set L. This design serves two complementary purposes. When *constructing* a typing derivation, the prover chooses L to be large enough that any witness outside it is guaranteed fresh, typically the union of all free names currently in scope, and then uses `exists_two_fresh` to produce concrete witnesses. When *using* a typing derivation, for example during inversion or induction, the universal quantifier allows the prover to instantiate x and y as whatever fresh names the current proof goal requires, rather than being tied to the specific names chosen during construction.

Explicit Disjointness Constraints. In the paper presentation of πLL, the disjointness of environments during operations like parallel composition or tensor is implicitly guaranteed by the structural properties of the merge operator for environments and hyperenvironments. However, because the Lean formalisation relies on total list concatenation, these disjointness requirements must be enforced explicitly at the rule level. Many typing constructors carry explicit disjointness and freshness premises to ensure that merging operations remain valid. For example, the `mix` rule requires a proof that the two hyperenvironments are mutually disjoint (`hD : 𝒢.disjoint ℋ`), and the `tensor` rule requires explicit proof that the subject channel x does not already exist in the continuations (`hF : x∉ Γ.names∧x∉ Δ.names`). While these local constraints clutter the inductive definitions compared to their paper counterparts, they are strictly necessary to ensure that the

formalised typing inductive preserve the partiality of the theoretical merge operator and its underlying monoid structure. The global metatheoretical consequences of these local constraint, namely, that well-typed processes inherently preserve context linearity, are detailed in **??**.

Structural invariants. The file `Judgement/Names.lean` establishes a key invariant for the typping system, namely `Typing_f_eq_names`. This lemma states that for any well-typed process, the free names of the process are exactly equal to the domain of the hyperenvironment:

$$n \vdash P :: \mathcal{G} \implies P.f = \mathcal{G}.names$$

This ensures both that the process has no untyped free channels and that the environment contains no unused channels, strict linearity in both directions.

Building on this, `Judgement/Linearity.lean` proves `Typing_preserves_disjointness`: any well-typed hyperenvironment is pairwise disjoint. Together these two results guarantee that typing derivations can only be constructed for well-formed, linear contexts.

Type weakening. `Judgement/Weakening.lean` establishes that typing is stable under type-level index shifting:

$$n \vdash P :: \mathcal{G} \implies \forall d\ c, n + c \vdash P \uparrow d, c :: \mathcal{G} \uparrow d, c$$

This is used whenever a process is placed under a new type binder, the existing derivation can be lifted to the new depth by shifting both the process and its hyperenvironment. The proof proceeds by structural induction on the typing derivation, with the `forall_` case requiring careful composition of the shift operations.

Example derivation. The `ax` constructor types the forwarding process $x \leftrightarrow y$, which links two dual endpoints directly:

```
| ax {x y : FPName} {A : Types} {n : Nat} (hneq : x ≠ y) :
    A.lc n ->
    Typing n (#x ↔ₚ #y) [x : A^⊥ :: [y : A]]
```

This requires only that $x \neq y$ and that the type A is locally closed at depth n. It produces a single-component hyperenvironment containing exactly the two endpoints with dual types, directly reflecting the axiom rule from Figure 1. As the simplest typing derivation in the system, it also serves as the base case in inductive proofs over the typing relation.

The full inductive definition of `Typing` is listed in **??**.

4.2 Semantics

The dynamic behavior of the calculus is formalised within the `Semantics/` namespace. This includes the definition of transition labels in `Semantics/Label.lean`, the core Labelled Transition System (LTS) over typing derivations in `Semantics/JudgementStep.lean`, and the induced transition systems over pure processes and environments (`ProcStep.lean` and `EnvStep.lean`).

The Typed Transition System. In standard presentations of process calculi, operational semantics are typically defined as a reduction relation over raw syntactic processes (e.g., $P \to Q$). However, because πLL enforces strict linear resource accounting, the formalisation defines the primary LTS intrinsically over the typing derivations itself.

The `TypingStep` inductive relation dictates how a well-typed process and its supporting hyperenvironment evolve during execution. A transition $\vdash P :: \mathcal{G} \xrightarrow{l} \vdash Q :: \mathcal{H}$ mathematically guarantees that the process P can perform an action l to become Q, and that the hyperenvironment $\mathcal{G}$ correctly

evolves into $\mathcal{H}$ to reflect the consumption or creation of linear resources. By baking the transition system directly into the typing judgements, the formalisation intrinsically links operational behavior to linear resource management.

Erasure Homomorphisms. While `TypingStep` provides the source of truth for the calculus, reasoning exclusively using the full derivations can become overwhelming due to the size and the constraints they enforce.

To alleviate this, the formalisation mechanises the erasure homomorphisms described in Section 2.6 and the environment equivalent described in Section 2.7. Following **??** we project out the process and environment of a `TypingStep` to define two independent LTSs:

- **`ProcStep`:** Extracted via the `proc` projection function, this relation models the syntactic evolution of the process ($P \xrightarrow{l} Q$) completely independent of its typing context.
- **`EnvStep`:** Extracted via the `env` projection function, this relation models the structural evolution of the hyperenvironment ($\mathcal{G} \xrightarrow{l} \mathcal{H}$) as resources are exchanged.

Because `ProcStep` and `EnvStep` are derived mathematically from `TypingStep`, they are guaranteed to be sound with respect to the linear typing rules. These projected relations serve as the primary foundational infrastructure for the subject reduction and progress theorems discussed in **??**.

Structural Invariants and Inversion. While `TypingStep` defines the valid transitions of the calculus, reasoning about these transitions mathematically requires a robust suite of structural guarantees. Just as the static typing judgements enforce exact names and context well-formedness, the dynamic transition relation must be proven to strictly preserve these linearity constraints across execution steps. Furthermore, because process calculi transitions are inherently non-deterministic, decomposing a `TypingStep` during proofs relies heavily on complex inversion lemmas (e.g., extracting the constituent environments from a tensor-parr communication). Because these inversion and linearity lemmas form the foundational proof architecture for the calculus, their specific mechanisation and application are deferred and discussed in detail alongside the main soundness theorems in **??**.

4.3 Session Fidelity

The final objective of the formalisation is to verify the core soundness properties of the πLL calculus, collectively referred to as Session Fidelity. Encapsulated within the `SessionFidelity/` namespace, this suite of proofs verifies that well-typed processes behave predictably, memory resources are strictly accounted for during execution, and the type system accurately restricts dynamic behavior.

Erasure Simulation. Before proving global soundness, the formalisation must mathematically validate the erasure homomorphisms defined in Section 4.2. The files `EnvStep.lean` and `ProcStep.lean` establish forward simulation theorems for the projected transition systems. Specifically, they prove that if a full derivation step occurs (`TypingStep`), the extracted process and environment can independently mimic that exact transition:

```
-- Simulation for Processes (proc : Der -> Proc)
-- Simulation for Environments (env : Der -> Env)
```

These simulation results confirm that `ProcStep` and `EnvStep` are sound abstractions of the underlying typed calculus, allowing the metatheory to reason about syntax and resources independently when necessary.

Subject Reduction. The cornerstone of session fidelity is the Subject Reduction theorem (often referred to as Type Preservation), mechanised in `SubjectReduction.lean`. The theorem states

that if a process P is well-typed under a hyperenvironment $\mathcal{G}$, and P takes an operational step to P' via action l, then there must exist a stepped hyperenvironment $\mathcal{H}$ such that P' is well-typed under $\mathcal{H}$, and $\mathcal{G}$ validly transitions to $\mathcal{H}$ via the same action l.

```
-- Put the subject reduction theorem here.
```

This theorem is proven by structural induction over the transitions relation, which requires reasoning about the extraction of a specific typing environment structure given a generic typing environment and the rule it originates from. This is the core of the formalisation where the hyperenvironment inversion lemmas (e.g., `Tensor_Parr_Res.lean` and `Bot_Res.lean`) are deployed. When two parallel processes communicate, the proof state only knows that their combined hyperenvironment steps. The inversion lemmas deconstruct the generic hyperenvironment, route the specific communication channels to the correct continuations, re-establish the explicit disjointness constraints (`hD` and `hF`), and reconstruct the modified hyperenvironment to satisfy the typing judgement for P'.

Typability and Coherent Evolution. While Subject Reduction guarantees the existence of a valid continuation state, `Typability.lean` elevates this guarantee back to the level of full typing derivations. The typability theorem proves that the structural evolution of the process and the environment step together coherently within the typed LTS:

```
-- put typability theorem here.
```

By bridging the gap between the untyped `ProcStep` and the strictly typed `TypingStep`, this theorem conclusively establishes Session Fidelity: a well-typed πMLL process will strictly obey its session protocols, consume linear resources correctly, and never evolve into an untypable or undefined state.

4.3.1 Semantics. The `Semantics` namespace formalises the operational behaviour of πMLL as three labelled transition systems. The primary LTS over typing derivations is defined in `JudgementStep/Basic.lean` as the inductive $\texttt{TypingStep}_{\mathtt{m}}$, the two other LTSs are $\texttt{ProcStep}_{\mathtt{m}}$ and $\texttt{EnvStep}_{\mathtt{m}}$ (found respectively in `ProcStep/Basic.lean` and `EnvStep/Basic.lean`), which are induced over processes and typing environments respectively, realising the erasure homomorphisms described in Section 2.5.

Labels. Transition labels are formalized in `Labels.lean`. While the operational semantics are restricted to the πMLL, the underlying action vocabulary defined by the `Act` inductive encompasses the full πLL calculus to facilitate future extensions. Consequently, `Act` maps directly to the complete set of operational actions and not just those introduced in Section 2.5. These include empty communication (`one`, `bot`), channel transmission (`tensor`, `parr`), type instantiation (`output`, `input`), and internal selection and server client usage (`muBrack`, `muParen`). The `Lbl` inductive then wraps these actions, extending them with the silent transition τ, a session link label, and the concurrent composition of two simultaneous actions.

```
inductive Mu : Type
  | L
  | R
  | DISP
  | DUP
  | USE

inductive Act : Type
```

```
  | one       (x : FPName)              -- x[]
  | bot       (x : FPName)              -- x()
  | tensor    (x y : FPName)            -- x[y]
  | parr      (x y : FPName)            -- x(y)
  | output    (x : FPName) (A : Types)  -- x[A]
  | input     (x : FPName) (A : Types)  -- x(A)
  | muBrack   (x : FPName) (μ : Mu)     -- x[μ]
  | muParen   (x : FPName) (μ : Mu)     -- x[μ]

inductive Lbl : Type
  | tau
  | link  (x y : FPName)
  | act   (p : Act)
  | par   (l l' : Act)
```

Each label carries computable sets of free names (`Lbl.f`) and introduced names (`Lbl.i`), used in the side conditions of the parallel and restriction rules. The well-formedness predicate `Lbl.WF` enforces that parallel labels introduce disjoint names, corresponding to the condition $i(l) \cap i(l') = \emptyset$ from Section 2.5.

The `HasBracket` and `HasParen` typeclasses from `Base.lean` are instantiated for `Act` and `Lbl`, allowing the same notation used for process prefixes to be reused for labels. For example, `x[[y]]` denotes both the output prefix in a process and the corresponding output label, keeping proof states readable.

4.3.2 Process Steps. The LTS over processes is defined in `Semantics/ProcStep/Basic.lean` as $\texttt{ProcStep}_{\texttt{m}}$. The action prefix rules are axioms with explicit freshness conditions. For example, the `tensor` rule requires the introduced name y to be fresh with respect to the subject x and the free names of the continuation:

```
| tensor {P : Proc} {x y : FPName} (hF : y ∉ {x} ∪ P.f) :
    ProcStepₘ (#x[[•]].P) (x[[y]]) P(( #y ))
```

Note that unlike the typing rules, `ProcStep` does not use co-finite quantification, the freshness condition is an explicit side condition on the introduced name y. This is by design: the process LTS operates purely on syntax without typing information, so there is no universal quantifier to exploit. The consequence is that `ProcStep` rules are not symmetric with respect to renaming, and inversion lemmas must account for the specific fresh name chosen. The two internal reduction rules are particularly notable. The `one_bot` rule handles channel closure:

```
| one_bot {P Q : Proc} {x y : FPName}
    (hx : x ∉ P.f) (hy : y ∉ P.f) (hneq : x ≠ y) :
    ProcStepₘ
```

The `tensor_parr` rule handles name-passing communication and is the most complex constructor in the formalisation, requiring six distinct freshness conditions across four names to ensure that the opened endpoints and transmitted names are all mutually distinct and do not appear in the continuations.

4.3.3 Judgement Steps and Environment Steps. The primary LTS is defined in `Semantics/JudgementStep/Basic.lean` as a relation between typing derivations:

```
inductive TypingStepₘ :
  Typing n P 𝒢 -> Lbl -> Typing n' P' 𝒢' -> Prop
```

Its structure closely mirrors $\texttt{ProcStep}_{\texttt{m}}$, with each constructor corresponding to the same rule at the level of typing derivations. The key difference is that every constructor carries a typing derivation as its subject and produces a new typing derivation as its conclusion, making the type-theoretic content of each step explicit.

$\texttt{EnvStep}_{\texttt{m}}$ is defined in `Semantics/EnvStep/Basic.lean` as the induced LTS on hyperenvironments, obtained by projecting $\texttt{TypingStep}_{\texttt{m}}$ to its environment component.

Design evolution: smart vs. dumb EnvStep. An earlier design attempted to define `EnvStep` as an independent inductive relation, mirroring the structure of `ProcStep` but operating on hyperenvironments instead of processes. This approach, preserved in `EnvStep\_old.lean`, added explicit disjointness conditions to the `par` and `syn` rules, conditions noted in the source as mimicking the corresponding side conditions in `ProcStep`. The intention was to make `EnvStep` self-contained, so that linearity properties could be proved directly by induction on `EnvStep` without reference to typing.

However, this design made `EnvStep` "smart" in the sense that it required side conditions that are only meaningful in the presence of typing information. This complicated the simulation proofs in `SessionFidelity`, since showing that `EnvStep` could mimic `JudgementStep` required discharging conditions that were not derivable from the structure of the derivation alone.

The final design instead defines `EnvStep` as a simple projection, a *dumb* relation that simply extracts the environment component of a `JudgementStep`. This delegates all well-formedness reasoning to the typing derivation, and the simulation results then follow directly from the projection.

ITEMS	TODO
Writing	Turn `Note` paragraphs into `remark` environments
Writing	Remove highlighting from process names not being part of a process in Background and BindingRepresentation
Writing	Fix content spread between the Binding Representation and Formalisation sections
Writing	Hold off on explaining rule side conditions too deeply in background; defer to formalisation chapter
Writing	In the metatheory/proof strategies section, discuss the structural vs. step induction dilemma for erasure
LaTeX	Fix Proposition 2.9 reference in the section on `Typing_f_eq_names`
LaTex	Make appendix of the full inductives from Lean

5 Future Work (WIP)

Operational semantics. While the current development provides a complete formalization of syntax, typing, and substitution for all constructs, the operational semantics and associated metatheory have been established for the multiplicative fragment not including the restriction rule. A natural direction for future work is to extend the operational semantics by completely incorporating the res-rule into the proofs of session fidelity. This would likely require additional reasoning about its behavior when interacting under an application of itself (basically an inversion lemma), and the any permutation its associated hyperenvironment might have, which could require the knowledge that a non-active component in a hyperenvironment is preserved across steps.

Extending the formalization to the full πLL system, would be the ultimate end goal, and would require the addition of a constructors for each of the rules in the non-multiplicative fragment of πLL to the already existing TypingStep inductive. An adaptation of all the lemmas pertaining to the properties of the TypingStep relation would need to be updated to account for these new constructors, and similar reasoning as for the res-rule is expected to be needed for each of the new rules to complete their respective cases in the proof for session fidelity.

Given that the foundational infrastructure for binding and substitution is already in place, this extension is expected to integrate smoothly albeit with the existing development maybe a little tediously when it comes to proving the various multi-thousand line permutations lemmas, which will likely be required. Completing these step would then result in a fully mechanized account of πLL, providing a tool for rigid validity checking of the already sound and existing theory

Structural Congruence. Another matter of interest would be addressing the mechanization of structural congruence. In the current implementation, the strict binary tree structure of the inductive types causes certain well-typed, theoretically reducible processes to become artificially stuck. This primarily manifests through the commutativity and associativity of parallel composition ($P \mid_p Q \equiv Q \mid_p P$ and $P \mid_p (Q \mid_p R) \equiv (P \mid_p Q) \mid_p R$), where the synchronization rule strictly expects interacting processes to be immediate siblings in a specific left-to-right syntactic order

Furthermore, mechanizing other standard congruences, such as Scope Extrusion, introduces unique challenges specific to the chosen syntax representation. By adopting a Locally Nameless (LN) approach, the traditional α-equivalence and fresh-name side conditions of Scope Extrusion are elegantly bypassed. However, they are replaced by the complexities of De Bruijn index arithmetic. Pulling a parallel process Q under a restriction binder ($\nu xy\, P \mid Q) \equiv \nu xy\, P \mid (\text{shift } 0\ 2\ Q)$ requires explicitly shifting the bound indices within Q to account for the increased binder depth. Because the typing hyperenvironments are mechanized using a List-based representation, associativity and commutativity of the contexts must be managed through explicit structural permutation rules.

Formally proving that these De Bruijn index manipulations distribute and commute safely across explicit list permutations and structural merges adds substantial proof overhead.

While replacing binary parallel composition with unordered multisets of processes is a common theoretical remedy to these rigidities, implementing this in a dependent type theory like Lean introduces severe dependent elimination issues. Because multisets are formalized as quotients of lists, indexing inductive typing judgments over them destroys syntax-based matching and inversion, drastically complicating proof search. Consequently, future iterations would be best served by maintaining the rigid binary abstract syntax tree and list-based environments. While the existing permutation rules allow for necessary flexibility in the typing context (background), adding a process congruence rule would make the transition system significantly heavier by collapsing the 1-to-1 mapping between syntax and behavior. Instead, extending the semantics with explicit symmetric structural rules (e.g., syn-symm) allows transitions modulo congruence while maintaining the syntactic tractability and index stability essential for manageable proofs in Lean.

References

Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) *(POPL '08)*. ACM, New York, NY, USA, 3–15. doi:10.1145/1328438.1328443

Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics*, Joe Hurd and Tom Melham (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 50–65.

Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.

Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23-26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33

Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2

N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0

Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/archives/fall2023/entries/logic-linear/

Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4

Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.

Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π-calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818

Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.